Хэрэглээний шинжлэх
ухаан, Инженерчлэлийн
сургууль

# EFFORT ESTIMATION-УР ДҮНТЭЙ БАЙДАЛ БОЛОН ЗАРЦУУЛАХ ХУГАЦААГ ТООЦОХ

**Намсрайдоржийн МӨНХЦЭЦЭГ**

**МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ**
**МУИС, Хэрэглээний шинжлэх ухаан инженерчлэлийн сургууль**
munkhtsetseg@seas.num.edu.mn

ICSI405 Програм хангамжийн төслийн удирдлага
2018 оны Намар

# Агуулга

- **Үр дүнтэй байдлыг тооцох- Тооцооллыг амжилттай болгох элементүүд**
- Тооцооллын асуудлууд
- Тооцооллын үндэс
  - Алхам 1:  Тооцооллыг эхлүүлэх
  - Алхам 2: Төслийн тооцоололд ашиглах EQF арга
  - Алхам 3: EQF-г төслийг удирдахад ашиглах
- Төслийг дуусгах хугацааг тооцох Albrecht Function Point Analysis-Function Point Analysis
- **COCOMO Model**

# Effort estimation-Ур ашигтай байдлыг тооцох

Програм хангамжийн инженерчлэлд дээрх үндсэн гол практикт гардаг асуудал бол програм хангамжийн үр ашигтай байдал юм. Үр ашигтай байдал гэдэгт хөгжүүлэлтийн цаг хугацааг чанартай үр бүтээлтэйг хэрхэн тооцох вэ гэдэг асуудал орно.

- “Програм хангамжийн практикт эмпирик нотолгооноос урьдчилан таамаглах чадвар илүү үр дүнтэй байдаг.
  Үүнийг батлах шаардлагагүй ... " гэж бичсэн байдаг.

# ҮР ДҮНТЭЙ БАЙДЛЫН ТООЦООЛОЛ

- Хүмүүсийн ажлын үр  дүнтэй байдлыг тооцох нь “BLACK ART.” гэж ярьдаг. Анх харахад мэдрэмжид үндэслэгдсэн юм шиг боловч энэ нь их субьектив үйл явц байна.
- Нэг ажлыг хийж гүйцэтгэхэд тодорхой хугацаа байх ёстой, уг хугацаатай дөхөж очоогүй тооцолол бол үнэн зөв биш юм.
- Ямар нэг төслийг бүтэцтэйгээр тооцоолж үзээгүй хүнд ирээдүйг урьдчилан тааварлах гэж байгаа юм шиг санагдана.
- Бэлтгэлгүй тооцооноос болж төсөл хугацаанаасаа хоцордог.

# Хугацааг амжилттай тооцох элементүүд

- Work Breakdown Structure (WBS)-сайн хуваах 10-20
- Ажил бүрийн хүчин чармайлтыг тооцно.-Өмнөх төсөлтэй харьцуулна.
- Хүчин чармайлтыг тооцох нь мэдээллийн үнэн зөв байдлаас их шалтгаална. 2 хүн нэг ажлыг өөр өөрөөр ойлгож өөр өөрөөр гүйцэтгэнэ гэсэн төлөвлөгөө гаргадаг. Жнь Цаг гаргах 1нь Командын түвшинд, нөгөөх нь Windows-н түвшинд
- Төслийн менежер тухайн ажлын алсын харааг зөв гаргах
- Төслийн менежер дараах асуултуудын дагуу хүчин чармайлтыг сайн тооцож байгаа мэдэж болно.

# ТААМАГЛАЛ НЬ ТООЦООЛЛЫГ ИЛҮҮ НАРИЙВЧЛАЛТАЙ БОЛГОДОГ.

- WBS-г багаараа хийдэг учраас wbs-г байгуулахдаа тухайн ажлыг дуусгах ажлыг тооцоолох хугацааг багаараа сайн ярилцаж байж тогтоох хэрэгтэй.
- Үүнийг таамаглал гэнэ.
- Таамаглал дэвшүүлэх нь туршлагаасаа хамаарч сайжрах ур чадвар юм. Гэсэн хэдий ч үүнийг үр дүнтэй болгох асуултууд байдаг.

# Үр дүнтэй байдлыг тооцоход авч үзэх асуудлууд

- Багийнханд мэддэг боловч ямар ч баримт бичигт бичигдээгүй төслийн зорилго бий юу?
- Ямар нэгэн тодруулах шаардлагатай ойлголт, нэр томъёо, тодорхойлолт бий юу?
- Үүнийг дагаж мөрдөх ёстой стандартууд байдаг боловч эдгээрийг дагаж мөрдөхөд үнэтэй байх уу?
- Энэ төслийн хөгжүүлэлт нь өмнөх төслүүдээс юугаараа ялгаатай вэ? Өмнө нь хийгээгүй шинэ ажлууд нэмэгдэх үү?
- Өмнө нь гаргасан технологи, архитектурын шийдлүүдийг хэрэглэх боломжтой юу?
- Энэ тооцооллыг буруу хийсэн байж болзошгүй өөр ямар нэг байгууллагад ямар өөрчлөлтүүд гарч байсан бол ?
- Төсөлд нөлөөлөх багийнхан зөвшилцөлд хүрээгүй ямар нэгэн асуудал  байгаа юу?

# Агуулга

- Үр дүнтэй байдлыг тооцох- Тооцооллыг амжилттай болгох элементүүд
- **Тооцооллын асуудлууд**
- Тооцооллын үндэс
  - Алхам 1:  Тооцооллыг эхлүүлэх
  - Алхам 2: Төслийн тооцоололд ашиглах EQF арга
  - Алхам 3: EQF-г төслийг удирдахад ашиглах
- Төслийг дуусгах хугацааг тооцох Albrecht Function Point Analysis-Function Point Analysis
- **COCOMO Model**

# Тооцооллын асуудлууд

Буруу тооцооллоос биелэшгүй төлөвлөгөө гарч ирдэг. Буруу тооцоолох шалтгаан

- Тооцоонд суурилсан өмнөх өгөгдөл дутмаг
- Тооцоолох туршлага дутмаг
- Тухайн төсөлд тооцоолох аргачлал, загвар, техник нь тохироогүй.
- Тооцооллын хүрээнд төслийн чухал үйл ажиллагаа, гарах үр дүнг оруулаагүй.
- Бодит бус хүлээлт эсвэл таамаглал
- Програм хангамжийн тооцоололд хамаарах шийдвэрлэх боломжгүй,тодорхойгүй байдлыг хүлээн зөвшөөрөөгүй зэргээс болдог.
- Дахин ашиглаж болох кодыг тооцох
- Өнгөрсөн өгөгдлийг ашиглах
- Төслийн алхам бүр дээр тооцоолол хийх
- Хэлэлцээр болгон дээр тооцоо хийх
- Төлөвлөсөн тооцооны хяналтыг хийх

# Агуулга

- Үр дүнтэй байдлыг тооцох- Тооцооллыг амжилттай болгох элементүүд
- Тооцооллын асуудлууд
- **Тооцооллын үндэс**
  - **Алхам 1: Тооцооллыг эхлүүлэх**
  - **Алхам 2: Төслийн тооцоололд ашиглах EQF арга**
  - **Алхам 3: EQF-г төслийг удирдахад ашиглах**
- Төслийг дуусгах хугацааг тооцох Albrecht Function Point Analysis-Function Point Analysis
- **COCOMO Model**

# Үр дүнтэй байдлыг тооцох

**Алхам 1: Үр дүнгийн тооцооллыг эхлүүлэх**

- **Тодорхой хугацаагаар 5.1-8.1**
- **5 хүн 3 сард хийнэ.**
- **5 сарын 1 гэхэд 90% дуусна**

**Алхам 2: Төслийн үр дүнгийн тооцооллыг хянахын тулд Төсөл дуусгах track –EQF-г байнга ярилцана. Төслийн менежер байнга төсөл хэзээ дуусах вэ? Гэдэг асуултыг багийнхнаас асуудаг байх**

Y тэнхлэг дээр гаргасан огноо, X тэнхлэг дээр асуулт тавьсан огноонуудыг график дээр дүрслээрэй. Энэ бол EQF юм.

**Алхам 3: Төслийн асуудлуудыг зохицуулахын тулд EQF-г ашиглах**

“"Төсөлд юу тохиолдсоныг мэдэж, ямар асуудлаар уулзах, ямар асуудлыг хурдлуулах, алийг нь өнгөрүүлэхээ мэдэж авах боломжтой."

- **EQF бол төслийн тодорхойгүй байдлыг харуулах, багт яагаад тухайн төсөлд тодорхойгүй байдал байгааг ойлгох гайхалтай арга юм.**

# Анхаарал хандуулсанд баярлалаа.

# Агуулга

- Үр дүнтэй байдлыг тооцох- Тооцооллыг амжилттай болгох элементүүд
- Тооцооллын асуудлууд
- Тооцооллын үндэс
  - Алхам 1:  Тооцооллыг эхлүүлэх
  - Алхам 2: Төслийн тооцоололд ашиглах EQF арга
  - Алхам 3: EQF-г төслийг удирдахад ашиглах
- **Төслийг дуусгах хугацааг тооцох Albrecht Function Point Analysis-Function Point Analysis**
- **COCOMO Model**

# Үр дүнг тооцох арга техникүүд

- Үр дүнг тооцох техникийг сонгоход дараах зүйлүүдийг харгалзан үзэх хэрэгтэй. Үүнд:
- Үр дүнг тооцох арга нь төсөөлтэй тохирч байгаа эсэх
- Шаардлагатай өгөгдөл нь эх сурвалжаас авагдаж байгаа эсэх
- Хэрэгжүүлж байгаа арга нь төслийн төлөвлөгдсөн үйл ажиллагаатай тохирч байгаа эсэх
- Хүснэгт№1-д төслийн шинж чанараас хамаарч өөр өөр үр дүнг тооцох арга техникийг ашиглахыг санал болгосон.

| Type of Project | Estimation Methods Used | Performance Factors Used for Adjusting Estimates |
|---|---|---|
| Development | • Size – KLOC/ FP/ Use Case Points<br>• Effort / schedule is derived from Size<br>• Activity Based Approach<br>• Number of requirements used to estimate calendar time for systems requirements specification, design and integration / system testing phases<br>• Parametric Model Approach<br>• Size – KLOC/ FP/ Use Case Points<br>• Effort / schedule – using COCOMO<br>• Effort / schedule / resources – using Knowledge Plan | • Productivity (person days required to produce a screen of given complexity)<br>• Application domain familiarity<br>• Familiarity with the technology |

| | | |
|---|---|---|
| Development (GUI) | Simple Estimating Relationships<br>• Size measure - no. of screens (of complexity Low/ Medium/ Very High) used for estimating effort / schedule in Construction (Coding/ Unit testing) phase<br>• Function Point<br>• Activity Based Approach<br>• Number of requirements used to estimate calendar time for systems requirements specification, design and integration / system testing phases<br>• Effort / schedule / resources - using Knowledge PLAN | • Productivity (person days required to produce a screen of given complexity)<br>• Application domain familiarity<br>• Familiarity with the technology |

| | | |
|---|---|---|
| Development (Reverse Engineering) | Analogy<br>• Size estimation in terms of total LOC<br>• Productivity (in LOC/day) from a previous reverse engineering project on similar platform and LOB was used<br>• Effort / schedule / resources - using Knowledge PLAN( Refer Estimation Binder under Information Sharing for more details on Knowledge PLAN ) | • The two projects were compared in terms of tool usage, knowledge transfer from customer, etc. and productivity value was adjusted accordingly. |

| | | |
|---|---|---|
| Development (Conversion) | Simple Estimating Relationships<br>• Size (in LOC) from the existing Source Code<br>• Effort estimation using the obtained size and the Conversion Language Table<br>• Effort / schedule / resources – using KnowledgePLAN | |
| Partial Life Cycle (Coding and Unit Testing) and Regular maintenance | Simple Estimating Relationships for coding new programs<br>• Size estimation in terms of LOC<br>• Use productivity (LOC/day) to get effort estimate<br>• Derive schedule based on effort estimate, customer's schedules and manpower available.<br>• Effort / schedule / resources – using Knowledge PLAN | • Productivity values from earlier work requests in the same project and / or from other similar projects are used |

| | | |
|---|---|---|
| Regular maintenance | Initial Team size estimation<br>Capture following parameters<br>• Database used<br>• # of Interfaces<br>• Total Number of programs / Screens / Reports<br>• Total number of Tables / Files<br>• Lines of code<br>• Tech Arch<br>• FTE (Full Time Employees) currently maintaining the system (includes all employees on the system)<br>• System Documentation quality<br>• Backlog in terms of man months<br>• Average enhancements per month<br>• Average production calls per month<br>Based on number of FTEs and the back log determine the number of resources needed to maintain the application | The following may be used to adjust the effort/ schedule estimates:<br>• Extent of testing required<br>• Interdependencies / impact on other modules<br>• Complexity of the modification<br>• Skills of the project team<br>• Wait time expected based on previous work requests data<br>• Rework expected based on previous work requests data |
| | Activity Based approach for modifying existing programs<br>• Size estimation in terms of programs / total LOC<br>• Identify the activities to be performed for the modification<br>• Use performance data from earlier work requests in the same project<br>• Derive schedule based on effort estimate, customer's schedules and manpower available.<br>• Use Knowledge PLAN to derive FTE required for support and corrective maintenance activity for a given period | |

# Агуулга

- Effort Estimation-Elements of a Successful Estimate
- Problems with Estimation
- Estimation Basis

    Phase 1: Create an Initial Estimate

    Phase 2: Track EQF to Understand the Project Estimate

    Phase 3: Use EQF to Manage Project Concerns

Estimation Techniques

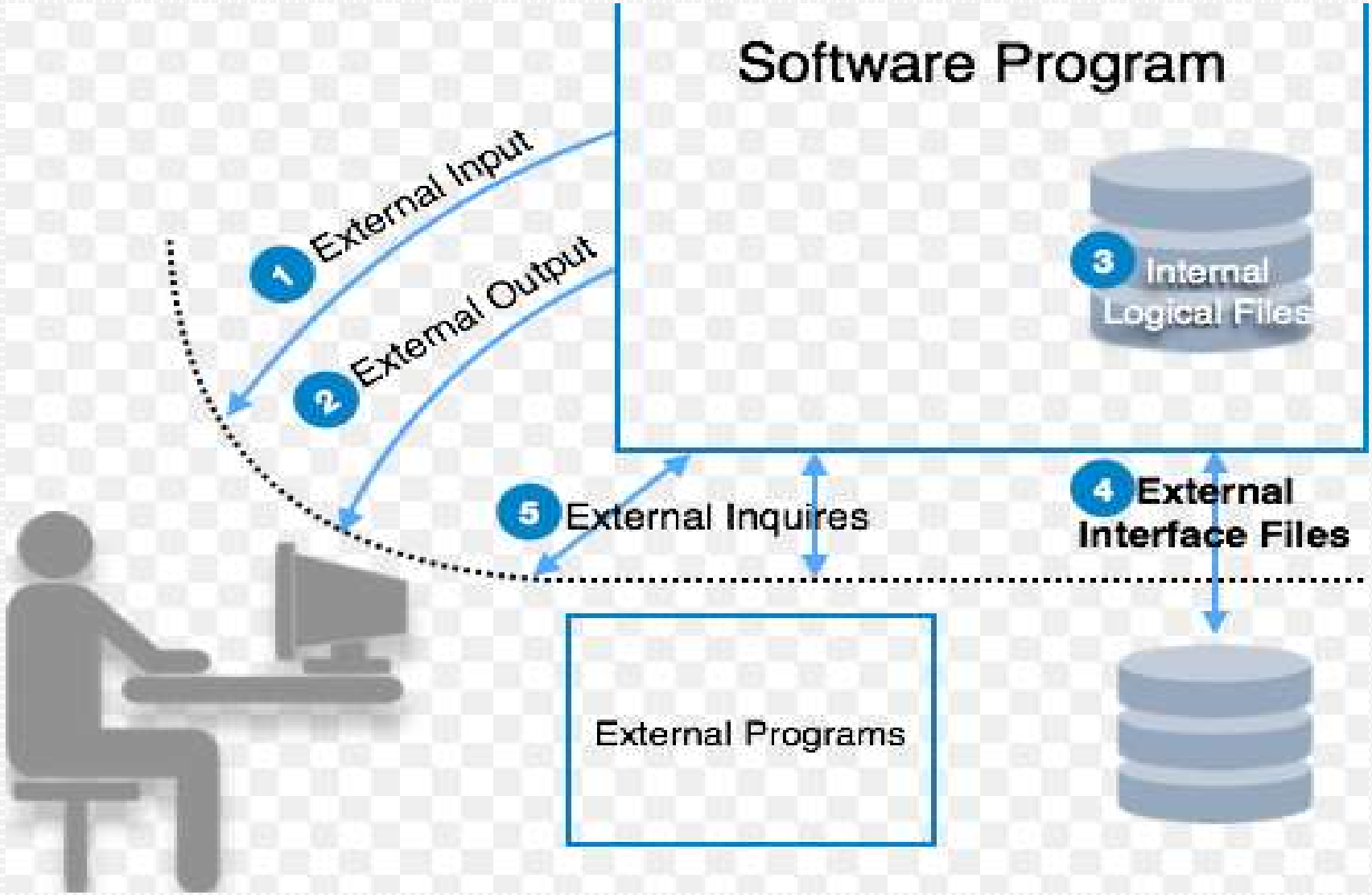
- **Albrecht Function Point Analysis-Function Point Analysis**
- Functions Mark II
- COCOMO Model
    - Brief Characteristic of the Method
    - Levels of the Model
    - Appraisal of the Model
    - Modes

# Albrecht Function Point Analysis

- **Function Point Analysis[1]**
- Function point analysis гэдэг нь асуудлыг шийдвэрлэх бүтэцлэгдсэн шинжилгээ хийх арга юм.
- Системийг хэрэглэгчийн хэрэгцээ шаардлага дээр тулгуурлан жижиг жижиг компонентуудад/үйл ажиллагаануудад хуваана.
- Function point буюу нэг жижиг компонент нь програм хангамжийг хэмжих нэгж болно.
- Энэь төслийн цар хүрээг хянах, tracking хийх гол хэрэгсэл болдог.
- Функцуудын тоо нь гэрээ үзэглэхэд чухал үүрэгтэй.
- **Энэ шинжилгээг туршлагатай мэргэжилтэн хийнэ.**
- 5 үндсэн класс, ерөнхий шинж чанаруудаас бүтнэ.

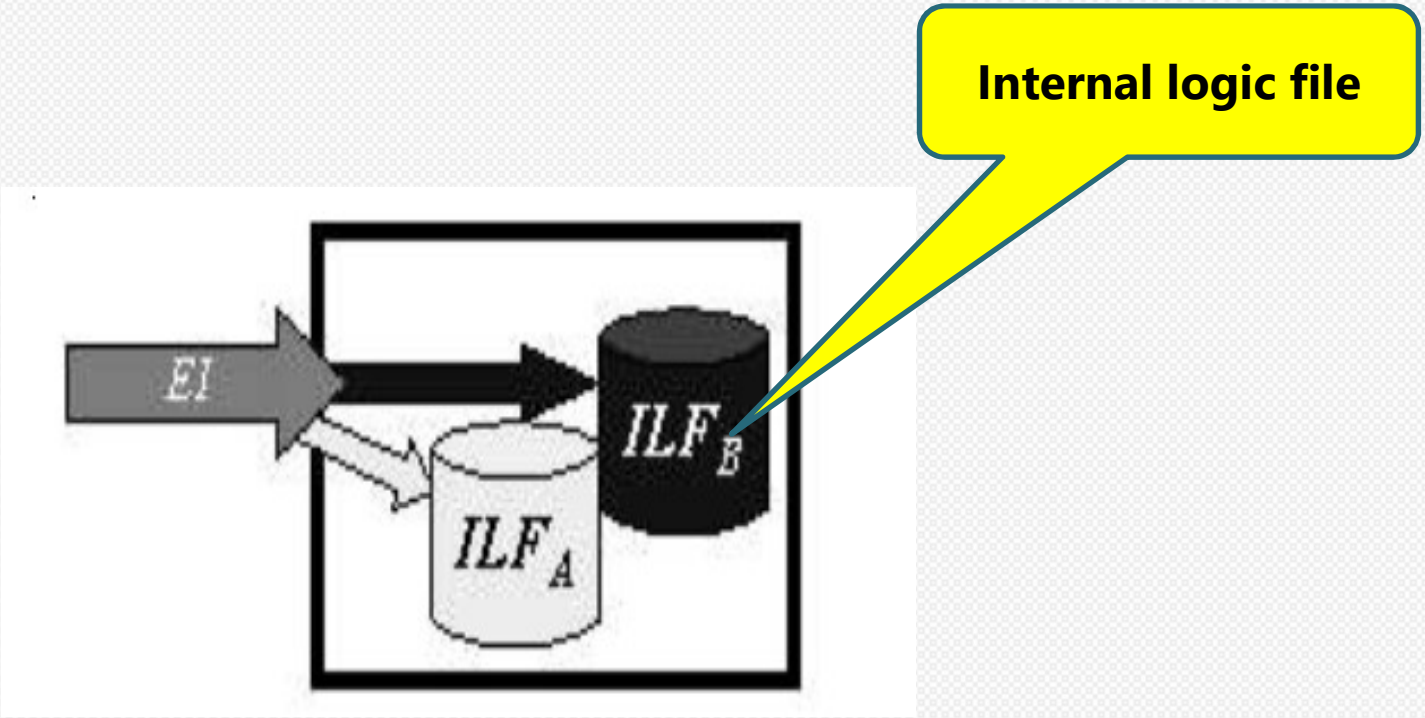
*Эх сурвалж: 1. Allan J. Albrecht in the mid 1970s.* ***IFPUG:*** International Function Point User Group

# FUNCTION POINT ANALYSIS



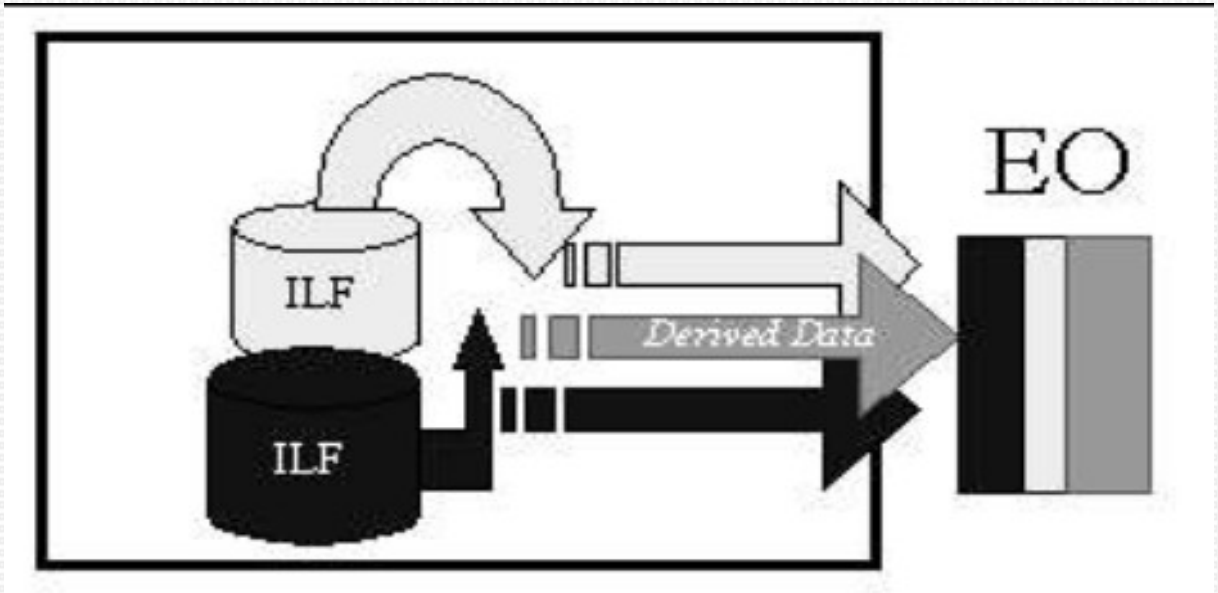
# Five Major Components- External Inputs (EI)

- **External Inputs (EI):**-Гадаад орчинтой харьцах цонх, интерфэйс мөн програм хангамж байна.
- Гаднаас орж ирж байгаа мэдээлэл дотоод файлуудын өөрчлөдөг.



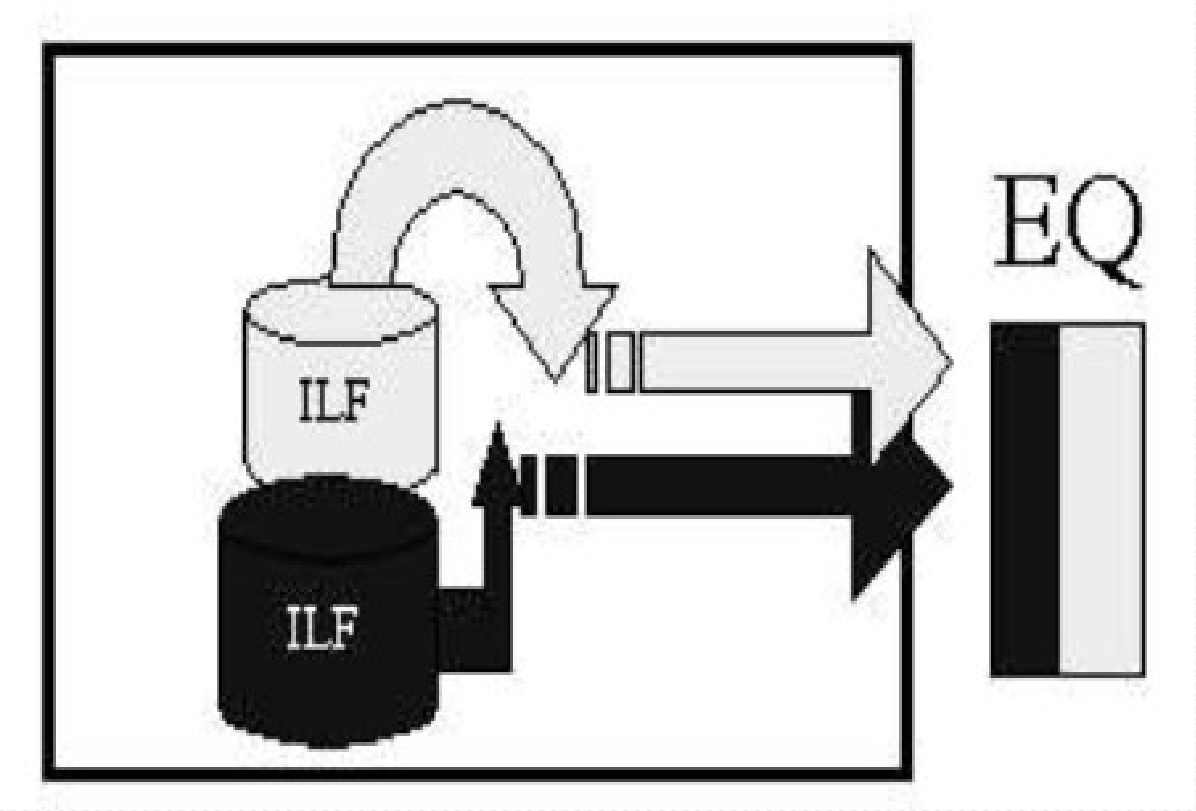
# Five Major Components- *External Outputs (EO)*

- ***External Outputs (EO):Програмын дотроос гадаад орчин руу гаргаж байгаа компонент буюу файл***
- ***Өгөгдөл гарахдаа мөн дотоод орчны мэдээллүүдийг өөрчилж болно.***



# FIVE MAJOR COMPONENTS- *EXTERNAL INQUIRY (EQ)*

- ***External Inquiry (EQ):-Гадны шаардлагаас шалтгаалан гарах, орох мэдээллийг боловсруулах компонент, файл.***
- ***Мэдээлэл гарах, ороходоо дотоод файлуудаб өөрчлөлт оруулахгүй.***



# Five Major Components- *Internal Logical Files (ILF), External Interface Files (EIF)*

- ***Internal Logical Files (ILF):*** Хэрэглэгчийн тодорхойлсон бүлэг дотоод логик файл. Энэ нь гаднаас орж ирэх мэдээллийг бүрэн боловсруулах копонент
- ***External Interface Files (EIF):*** Гаднаас тухайн application-д лавлагаа байдлаар хэрэгтэй хэрэглэгчийн тодорхойлсон бүлэг логик файл буюу компонент.
- Нэг программын дотоод файл нь өөр нэг программын гадаад файл болдог.
- Бүх компонентууд нь (EI's, EO's, EQ's,ILF's or EIF's)-н аль нэгэнд хамаарагдана. Эрэмбээрээ ялгагдана. Жнь: транзакшин буюу гүйлгээний мэдээлэл (EI's, EO's, EQ's,ILF's) нь нийт мэдээллийг тоог олох, өгөгдлийн төрлийг олох зэргээс илүү эрэмбэтэй.
- Эрэмбэ нь low, Average, High гэсэн утгатай байна.

| FTR's | DATA ELEMENTS | | |
|---|---|---|---|
| | 1-4 | 5-15 | > 15 |
| 0-1 | Low | Low | Ave |
| 2 | Low | Ave | High |
| 3 or more | Ave | High | High |

file types referenced (FTR) (combined unique input and output sides)

INTERNAL LOGIC FILE-ILF-н жишээ

*Transaction-жишээ*

*Shared EO and EQ Table-н жишээ*

Ашиглагдаж байгаа элементийн тоо

| Rating | VALUES | | |
|---|---|---|---|
| | EO | EQ | EI |
| Low | 4 | 3 | 3 |
| Average | 5 | 4 | 4 |
| High | 7 | 6 | 6 |

| FTR's | DATA ELEMENTS | | |
|---|---|---|---|
| | 1-5 | 6-19 | > 19 |
| 0-1 | Low | Low | Ave |
| 2-3 | Low | Ave | High |
| > 3 | Ave | High | High |

| RET's | DATA ELEMENTS | | |
|---|---|---|---|
| | 1-19 | 20 - 50 | > 50 |
| 1 | Low | Low | Ave |
| 2-5 | Low | Ave | High |
| > 5 | Ave | High | High |

The rating is based upon the total number of unique (combined unique input and out sides) data elements (DET)

A Record Element Type is a user recognizable subgroup of data elements within an ILF or EIFA Record Element Type is a user recognizable subgroup of data elements within an ILF or EIF

# Компонентүүдийн мэдээллүүдийг нэгтгэн бодох

| Type of Component | Complexity of Components | | | |
|---|---|---|---|---|
| | Low | Average | High | Total |
| External Inputs | ___ x 3 = ___ | ___ x 4 = ___ | ___ x 6 = ___ | |
| External Outputs | ___ x 4 = ___ | ___ x 5 = ___ | ___ x 7 = ___ | |
| External Inquiries | ___ x 3 = ___ | ___ x 4 = ___ | ___ x 6 = ___ | |
| Internal Logical Files | ___ x 7 = ___ | ___ x 10 = ___ | ___ x 15 = ___ | |
| External Interface Files | ___ x 5 = ___ | ___ x 7 = ___ | ___ x 10 = ___ | |
| | | Total Number of Unadjusted Function Points | | ___ |
| | | Multiplied Value Adjustment Factor | | ___ |
| | | Total Adjusted Function Points | | ___ |

**Нийт компонентийн элементийн тоо**

14 General System Characteristics (GSC)-р тодорхойлогдоно.

# General System Characteristics (GSC)-Системийн ерөнхий шинж чанар

| General System Characteristic | | Brief Description |
|---|---|---|
| 1. | Data communications | How many communication facilities are there to aid in the transfer or exchange of information with the application or system? |
| 2. | Distributed data processing | How are distributed data and processing functions handled? |
| 3. | Performance | Was response time or throughput required by the user? |
| 4. | Heavily used configuration | How heavily used is the current hardware platform where the application will be executed? |
| 5. | Transaction rate | How frequently are transactions executed daily, weekly, monthly, etc.? |
| 6. | On-Line data entry | What percentage of the information is entered On-Line? |
| 7. | End-user efficiency | Was the application designed for end-user efficiency? |
| 8. | On-Line update | How many ILF's are updated by On-Line transaction? |
| 9. | Complex processing | Does the application have extensive logical or mathematical processing? |
| 10. | Reusability | Was the application developed to meet one or many user's needs? |
| 11. | Installation ease | How difficult is conversion and installation? |
| 12. | Operational ease | How effective and/or automated are start-up, back-up, and recovery procedures? |
| 13. | Multiple sites | Was the application specifically designed, developed, and supported to be installed at multiple sites for multiple organizations? |
| 14. | Facilitate change | Was the application specifically designed, developed, and supported to facilitate change? |

Factor (VAF)-r rapрана.IF

# 0..5 ХООРОНД ЖИН ӨГЧ ҮНЭЛНЭ.

# ~~0=ХАМААРАЛГҮЙ … 5 = ЧУХАЛ~~

| | |
|---|---|
| 1 | Reliable backup and recovery required |
| 2 | Data communications |
| 3 | Distributed functions |
| 4 | High Performance (time efficiency) required |
| 5 | ? Heavily used configuration |
| 6 | On-line (interactive) data entry |
| 7 | Must be easy to use |
| 8 | On-line update |
| 9 | Complex user interface |
| 10 | Complex process |
| 11 | ? Reusability |
| 12 | Ease of installation |
| 13 | Multiple installation sites |
| 14 | Easy to modify |

TCF (Technical Complexity Factor)

$$MVAF = TCF_1*(0..5)+TCF_2*(0..5)+.....$$

TCF (Technical Complexity Factor)

MVAF (Multified Value Adjusting Factor)

# Albrecht Function Point Analysis

Function Points-FP =UFP × MVAF*0.01

UFC (Unadjusted Function Count

MVAF (Multified Value Adjusting Factor)

Өөрийн компаниас хамааран тохируулна.

**(One Function Point = 2 days !)**

# ALBRECHT APPROACH SUFFERS FROM THE FOLLOWING WEAKNESSES:

- It is often difficult to identify the components of an application. For example, what is a
- logical file? In 1984, there was insufficient guidance to make these determinations.
- ▪ Albrecht had assigned weights to function point components based on "debate and trial."
- ▪ The above two criticisms were also leveled at the identification and weighting of Value
- Adjustment Factors.
- ▪ Albrecht did not provide a means of accounting for internal complexity. This is the same
- problem regarding algorithms that the feature points technique was developed to address.
- ▪ When small systems are combined into larger applications, Albrecht's approach makes
- the total function point count less than the sum of the components.

# FUNCTIONS MARK II

- MK II (Mark 2) Method is one of the software sizing methods in functional point group of measurements. This is a method for analysis and measurement of information processing applications based on end user functional view of the system. The MK II Method (ISO/IEC 20968
- Software engineering—MK II Function Point Analysis—Counting Practices Manual) is one of five currently recognized ISO standards for functionally sizing software.

# FUNCTIONS MARK II - CONCEPT

- Symons decomposes the application being counted into a collection of logical transactions.
- Each transaction consists of an input, a process and an output. For each transaction, Unadjusted Function Points (UFP) become a function of the number of input data element-types, entity-types referenced
- and output data element-types. The UFPs for the entire system are then summed.

# Анхаарал хандуулсанд баярлалаа